13 Distributed Monitoring

The modern economy depends on the continual operation of IT infrastructure. Such infrastructure contains hardware, distributed services, and network resources. These components are interlinked in such infrastructure, making it challenging to keep everything functioning smoothly and without application downtime.

It's challenging to know what's happening at the hardware or application level when our infrastructure is distributed across multiple locations and includes many servers. Components can run into failures, response latency overshoot, overloaded or unreachable hardware, and containers running out of resources, among others. Multiple services are running in such an infrastructure, and anything can go awry.

When one of the services goes down, it can be the reason for other services to crash, and as a result, the application is unavailable to users. If we don't know what went wrong early, it could take us a lot of time and effort to debug the system manually. Moreover, for larger services, we need to ensure that our services are working within our agreed service-level agreements. We need to catch important trends and signals of impending failures as early warnings so that any concerns or issues can be addressed.

Monitoring helps in analyzing such complex infrastructure where something is constantly failing. Monitoring distributed systems entails gathering, interpreting, and displaying data about the interactions between processes that are running at the same time. It assists in debugging, testing, performance evaluation, and having a bird's-eye view over multiple services.

We've divided the distributed monitoring system design into the following chapters and lessons:

Distributed Monitoring

Introduction to Distributed Monitoring: Learn why monitoring in a distributed system is crucial, how costly downtime is, and the types of monitoring.

Prerequisites for a Monitoring System: Explore a few essential concepts about metrics and alerting in a monitoring system.

Monitoring Server-side Errors

Designing a Monitoring System: Define the requirements and high-level design of the monitoring system.

A Detailed Design of the Monitoring System: Go into the details of designing a monitoring system, and explore the components involved.

Visualize Data in a Monitoring System: Learn a unique way to visualize an enormous amount of monitoring data.

Monitor Client-side Errors

Focus on Client-side Errors: Get introduced to client-side errors and why it's important to monitor them.

Design a Client-side Monitoring System: Learn to design a system that monitors the client-side errors.

Need for monitoring

Let's go over how the failure of a single service can affect the smooth execution of related systems. To avoid cascading failures, monitoring can play a vital role with early warnings or steering us to the root cause of faults.

Let's consider a scenario where a user uploads a video, `intro-to-system-design`, to YouTube.

The `UI` service in server `A` takes the video information and gives the data to service 2 in server B.

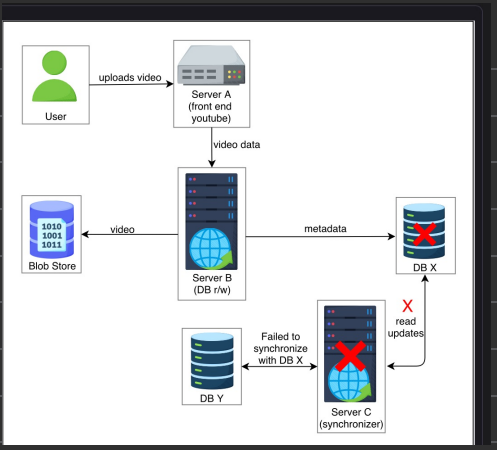
Service 2 makes an entry in the database and stores the video in blob storage.

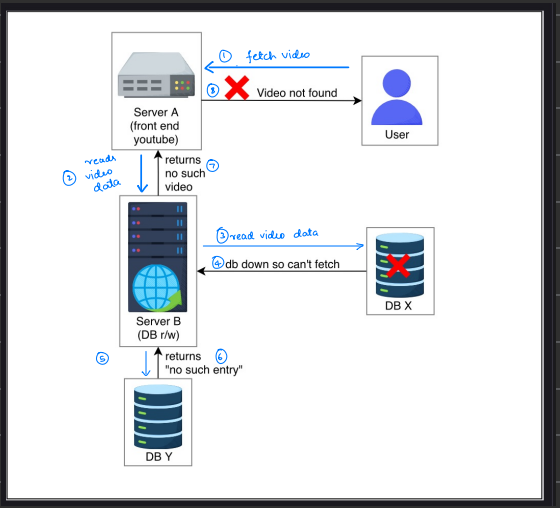
Another service, 3, in server C manages the replication and synchronization of databases X and Y.

In this scenario, service 3 fails due to some error, and service 2 makes an entry in the database X.

The database X crashes, and the request to fetch a video is routed to database Y.

The user wants to play the video `intro-to-system-design`, but it will give an error of "Video not found…"





Now, what if a learner makes a request and it never reaches the servers of Educative. How will Educative know that a learner is facing an issue?

With the above examples, we can divide our monitoring focus into two broad categories of errors:

Server-side errors: These are errors that are usually visible to monitoring services as they occur on servers. Such errors are reported as error 5xx in HTTP response codes.

Client-side errors: These are errors whose root cause is on the client-side. Such errors are reported as error 4xx in HTTP response codes. Some client-side errors are invisible to the service when client requests fail to reach the service.

Monitoring: metrics and alerting

A good monitoring system needs to clearly define what to measure and in what units (metrics). The monitoring system also needs to define threshold values of all metrics and the ability to inform appropriate stakeholders (alerts) when values are out of acceptable ranges. Knowing the state of our infrastructure and systems ensures service stability. The support team can respond to issues more quickly and confidently if they have access to information on the health and performance of the deployments. Monitoring systems that collect measurements, show data, and send warnings when something appears wrong are helpful for the support team.

To further understand metrics, alerts, and their connection with monitoring, we'll go over their significance, their potential benefits, and the data we might want to keep track of.

Prerequisites of a Monitoring System

- Monitoring: metrics and alerting
- Metrics
  - Populate the metrics
  - Persist the data
  - Application metrics
- Alerting

What are the conventional approaches to handle failures in IT infrastructure?

Hide Answer

The two conventional approaches to handling IT infrastructure failures are the reactive and proactive approaches.

In the reactive approach, corrective action is taken after the failure occurs. In this approach, even if DevOps takes quick action to find the cause of the error and promptly handle the failures, it causes downtime. As a result, there will be system downtime in the reactive approach, which is generally undesirable for continuously running applications.

In a proactive approach, proactive actions are taken before failure occurs. Therefore, it prevents downtimes and associated losses. The proactive approach works on predicting system failures to take corrective action to avoid the failure. This approach offers better reliability by preventing downtime.

In modern services, completely avoiding problems is not possible. Something is always failing inside huge data centers and network deployments. The goal is to find the impending problems early on and design systems in such a way that service faults are invisible to the end users.

Me

Metrics

Metrics objectively define what we should measure and what units will be appropriate. Metric values provide an insight into the system at any point in time For example, a web server's ability to handle a certain amount of traffic per second or its ability to join a pool of web servers are examples of high-level data correlated with a component's specific purpose or activity. Another example can be measuring network performance in terms of throughput (megabits per second) and latency (round-trip time). We need to collect values of metrics with minimal performance penalty. We may use user-perceived latency or the amount of computational resources to measure this penalty.

Values that track how many physical resources our operating system uses can be a good starting point. If we have a monitoring system in place, we don't have to do much additional work to get data regarding processor load, CPU statistics like cache hits and misses, RAM usage by OS and processes, page faults, disc space, disc read and write latencies, swap space usage, and so on. Metrics provided by many web servers, database servers, and other software help us determine whether everything is running smoothly or not.

We use the `top` command to view Linux processes. Running this command opens an interactive view of the running system containing a summary of the system and a list of processes or threads. The default view has the following:

On the top, we can see how long the machine has been turned on, how many users are logged in, and the average load on the machine for the past few minutes.

On the next line, we can see the state (`running`, `sleeping`, or `stopped`) of tasks running on the machine.

Next, we have the CPU consumption values.

Lastly, we have an overview of physical memory how much of it is free, used, buffered, or available.

Now, let's take the following steps to see a change in the CPU usage:

Quit by entering `q` in the terminal.

Run `nohup ./script.sh &>/dev/null &`. This script has an infinite loop running in it, and running the command will execute the script in the background.

Run the `top` command to observe an increase in CPU usage.

Populate the metrics

The metrics should be logically centralized for global monitoring and alerting purposes. Fetching metrics is crucial to the monitoring system. Metrics can either be pushed or pulled into a monitoring system, depending on the preference of the user.

Here, we confront a fundamental design challenge now: Do we utilize push or pull? Should the server proactively send the metrics' values out, or should it only expose an endpoint and wait reactively for an inquiry?

In pull strategy, each monitored server merely needs to store the metrics in memory and send them to an exposed endpoint. The exposed endpoint allows the monitoring application to fetch the metrics itself. Servers sending too much data or sending data too frequently can't overload the monitoring system. The monitoring system will pull data as per its own schedule.

Persist the data

Figuring out how to store the metrics from the servers that are being monitored is important. A centralized in-memory metrics repository may be all that's needed. However, for a large data center with millions of things to monitor, there will be an enormous amount of data to store, and a time-series database can help in this regard.

Time-series databases help maintain durability, which is an important factor. Without a historical view of events in a monitoring system, it isn't very useful. Samples having a value of time stamp are stored in chronological sequence. So, a whole metric's timeline can be shown in the form of a time series.

Application metrics

We may need to add code or APIs to get the application metrics we care about for other components, notably our own applications. We embed logging or monitoring code in our applications, called **code instrumentation**, to collect information of interest.

Looking at metrics as a whole can shed light on how our systems are performing and how healthy they are. Monitoring systems employ these inputs to generate a comprehensive view of our environment, automate responses to changes like commissioning more EC2 instances if the applications' traffic increases, and warn humans when necessary. These system metrics are system measurements that allow analyzing historical trends, correlations, and changes in performance, consumption, or error rates.

Alerting

Alerting is the part of a monitoring system that responds to changes in metric values and takes action. There are two components to an alert definition: a metrics-based condition or threshold, and an action to take when the values fall outside of the permitted range.